# MEDIQUEUE: HOSPITAL MANAGEMENT SYSTEM

**A PROJECT REPORT**

***Submitted by***

Ishita Srivastava (24BCS10823)

Priyanshi Gupta (24BCS10819)

***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

**IN**

COMPUTER SCIENCE AND ENGINEERING




**Chandigarh University**

October-2026

# BONAFIDE CERTIFICATE

Certified that this project report **"MEDIQUEUE: HOSPITAL MANAGEMENT SYSTEM"** is the bonafide work of **"Ishita Srivastava (24BCS10823) and Priyanshi Gupta (24BCS10819)"** who carried out the project work under my supervision.

**SIGNATURE**

Gagandeep Singh

**HEAD OF THE DEPARTMENT**

Department of Computer Science and Engineering

Chandigarh University, Gharuan, Mohali, Punjab – 140413

**SIGNATURE**

Venkatesh K

**SUPERVISOR**

Technical Trainer

Department of Computer Science and Engineering

Chandigarh University, Gharuan, Mohali, Punjab – 140413

Submitted for the project viva-voce examination held on _____

# ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported us during the development of this project, “MediQueue: Hospital Management System”, carried out as part of the subject *Fullstack Development – II (24CSP-304)*.

We are deeply thankful to our project supervisor, Venkatesh K, for the continuous guidance, constructive feedback and encouragement that shaped this work from the first concept discussion to the final report. The supervisor’s insistence on clean design, proper validation and well-documented code helped us build a project that we are proud of.

We also thank the Head of the Department, Gagandeep Singh, and the faculty members of the Department of Computer Science and Engineering, Chandigarh University, for providing the laboratory facilities, software environment and academic atmosphere required to complete the project.

We are grateful to the open-source communities behind React, Node.js, Express, MongoDB and Mongoose, whose freely available tools and documentation made it possible to build a complete web application with open technologies.

Finally, we thank our families and friends for their patience, motivation and unwavering support throughout the project.

**Ishita Srivastava (24BCS10823)**

**Priyanshi Gupta (24BCS10819)**

# TABLE OF CONTENTS

**Acknowledgement** ..... **i**
**Table of Contents** ..... **ii**
**List of Figures** ..... **iii**
**List of Tables** ..... **iv**
**Abstract** ..... **v**
**Graphical Abstract** ..... **vi**
**Abbreviations** ..... **vii**
**Symbols** ..... **viii**
**Chapter 1: Introduction** ..... **1**
1.1. Client Identification / Need Identification / Identification of Relevant Contemporary Issue ..... 1
1.2. Identification of Problem ..... 2
1.3. Identification of Tasks ..... 3
1.4. Timeline ..... 3
1.5. Organization of the Report ..... 4
**Chapter 2: Design Flow/Process** ..... **5**
2.1. Concept Generation, Evaluation & Selection of Specifications/Features ..... 5
2.2. Design Constraints ..... 5
2.3. Analysis and Feature Finalization Subject to Constraints ..... 6
2.4. Design Flow ..... 7
2.5. Design Selection ..... 8
2.6. Implementation Plan/Methodology ..... 9
**Chapter 3: Results Analysis and Validation** ..... **15**
3.1. Implementation of Solution ..... 15
3.2. Output Screens ..... 17
3.3. Testing, Characterization and Data Validation ..... 20
**Chapter 4: Conclusion and Future Work** ..... **24**
4.1. Conclusion ..... 24
4.2. Future Work ..... 24
**References** ..... **26**
**Appendix-1: MediQueue – Data Model and Core Logic (Source Listings)** ..... **28**
**User Manual** ..... **30**
U1. Software required ..... 30
U2. Preparing the database ..... 30
U3. Configuring the backend ..... 30
U4. Starting the application ..... 30
U5. Using the application ..... 31
U6. Troubleshooting ..... 31
**Achievements** ..... **33**

# LIST OF FIGURES

Figure 1.1: Gantt chart of the project timeline ..... 3
Figure 2.1: Block diagram of Design A (React SPA + Express REST API) ..... 8
Figure 2.2: Block diagram of Design B (server-rendered Express + EJS) ..... 8
Figure 2.3: Layered architecture of MediQueue ..... 10
Figure 2.4: Database schema of the collections ..... 11
Figure 2.5: Flowchart of the bed allocation algorithm ..... 12
Figure 2.6: Flowchart of request handling and validation ..... 13
Figure 3.1: Project directory structure ..... 16
Figure 3.2: Dashboard with bed counts, actions and ward occupancy ..... 18
Figure 3.3: Waiting queue ranked by priority score ..... 18
Figure 3.4: Patient list with search, filters, paging and actions ..... 19
Figure 3.5: Add patient form showing a validation message ..... 19
Figure 3.6: Allocation log ..... 20

# LIST OF TABLES

Table 1.1: Team members and primary responsibilities ..... 4
Table 2.1: Evaluation of candidate features ..... 5
Table 2.2: Design constraints and how they were addressed ..... 6
Table 2.3: Comparison of Design A and Design B ..... 9
Table 2.4: REST API endpoints ..... 13
Table 3.1: Tools and technologies used ..... 15
Table 3.2: Functional test cases ..... 20
Table 3.3: Validation, boundary and negative test cases ..... 21
Table 3.4: Priority score and error-handling test cases ..... 22
Table 3.5: Performance of the priority computation ..... 23
Table 3.6: Security analysis ..... 23

# ABSTRACT

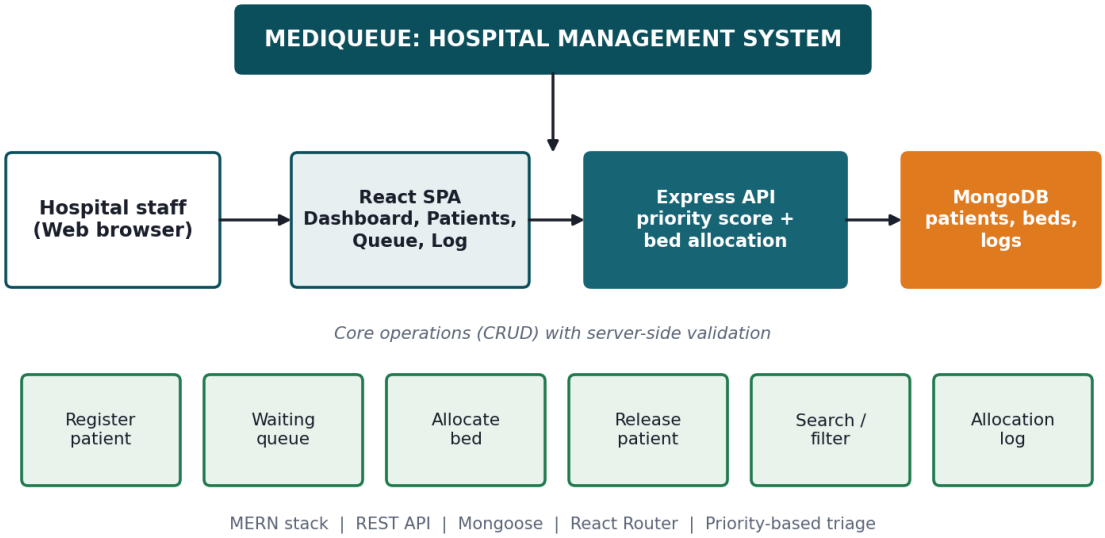
Hospitals often receive patients faster than beds become free, and the order in which waiting patients are admitted is frequently decided by hand, from notes, phone calls and memory. This is slow, hard to audit and can disadvantage patients who are seriously ill but quiet. The *MediQueue: Hospital Management System* presented in this report is a web application that ranks waiting patients by a transparent priority score and places each one in the most suitable ICU, Emergency or General ward bed.

The system is built with the MERN stack taught in the Fullstack Development – I course. A React single-page application (Vite, React Router, hooks and plain CSS) provides the dashboard, patient registration, search and filtering, the waiting queue and the allocation log. An Express REST API on Node.js handles routing, request validation, logging and error handling. MongoDB, accessed through Mongoose schemas with field-level validation and references, stores patients, beds and the allocation history. The priority score combines severity, waiting time, distance from the hospital and age; severe cases are offered ICU beds first, and the system falls back to other wards when a ward is full.

Two alternative designs, a React single-page application with a REST API and a server-rendered Express application, were compared before the first was selected. Forty-four automated test cases covering the API, validation rules, priority score, ward fallback and error handling all passed. The report also lists the limitations of the current version, mainly that user authentication is not enforced, and the improvements planned as future work.

**Keywords:** Hospital Management, Bed Allocation, Priority Score, MERN Stack, React, Node.js, Express, MongoDB, Mongoose, REST API.

# GRAPHICAL ABSTRACT




Hospital staff use the React pages in the browser. Requests reach a single Express API, which validates input, computes the priority score and allocates beds. Mongoose reads and writes the patients, beds and allocation log in MongoDB.

# ABBREVIATIONS

**Table A1: List of abbreviations used in this report**

| Abbreviation | Full form |
|---|---|
| API | Application Programming Interface |
| CORS | Cross-Origin Resource Sharing |
| CRUD | Create, Read, Update, Delete |
| CSS | Cascading Style Sheets |
| DPDP | Digital Personal Data Protection (Act, 2023) |
| EJS | Embedded JavaScript templates |
| ESI | Emergency Severity Index |
| HTML | HyperText Markup Language |
| HTTP | HyperText Transfer Protocol |
| ICU | Intensive Care Unit |
| JSON | JavaScript Object Notation |
| JSX | JavaScript XML |
| JWT | JSON Web Token |
| MERN | MongoDB, Express.js, React, Node.js |
| NoSQL | Not only SQL (non-relational database) |
| ODM | Object Document Mapper |
| REST | Representational State Transfer |
| SPA | Single-Page Application |
| UI | User Interface |
| XSS | Cross-Site Scripting |

# SYMBOLS

The following symbols and notations are used in the formulas, diagrams, code listings and tables of this report.

**Table S1: Symbols and notations**

| Symbol | Meaning |
|---|---|
| S | Severity of the patient (1 to 5) |
| W | Waiting time in minutes |
| D | Distance from the hospital in kilometres |
| A | Age bonus (10 for age 5 or under or 65 or over, otherwise 0) |
| P | Priority score of a patient |
| :id | Route parameter, for example /api/patients/:id |
| ? | Start of the query string, for example ?search=ali |
| $regex | MongoDB operator for regular-expression matching |
| -> | Reference from one collection to another |
| Rounded rectangle (green) | Start or end terminal in a flowchart |
| Rectangle | Process step in a flowchart or block in a diagram |
| Diamond | Decision in a flowchart |

# CHAPTER 1.
# INTRODUCTION

## 1.1. Client Identification / Need Identification / Identification of Relevant Contemporary Issue

### 1.1.1. Client identification

The intended client of this project is the admissions or bed-management desk of a mid-sized hospital: the nursing supervisors and administrative staff who decide, hour by hour, which waiting patient receives which bed. The people affected indirectly are the patients and their families, the doctors who depend on timely admission, and the hospital management, who need an accurate picture of bed occupancy. The same system can also be used by a clinic, a nursing home or a college health centre that has a small, fixed number of beds.

### 1.1.2. Need identification

In many hospitals the waiting list is kept on paper, on a whiteboard or in a spreadsheet, and the next patient is chosen from memory or by whoever asks most urgently. From an analysis of this way of working, the following needs were identified:

- a single, shared record of which patients are waiting and which beds are free, so that everyone sees the same picture;
- an objective and explainable way to decide who is admitted first, based on how ill the patient is and how long the patient has waited;
- automatic matching of a patient to the right type of bed (ICU, Emergency or General), with a sensible fallback when a ward is full;
- a record of every allocation so that decisions can be reviewed later;
- quick search, filtering and correction of patient records; and
- a simple interface that staff can use with very little training.

These needs describe a *coordination problem*: the information exists but is scattered and out of date. A lightweight web application that runs in an ordinary browser is a suitable remedy because it needs no special hardware.

### 1.1.3. Relevant contemporary issues

Several contemporary developments make a reliable digital bed-allocation tool important:

- **Overcrowding and waiting times.** Emergency departments report long waits because patients cannot be moved quickly into inpatient beds. Structured triage tools such as the Emergency Severity Index (Gilboy et al., 2020) and the Manchester Triage System (Mackway-Jones et al., 2014) exist so that the sickest patients are treated first, which is the same principle used in this project.
- **Digital health in India.** The Ayushman Bharat Digital Mission, run by the National Health Authority, encourages hospitals to replace paper processes with digital records and services.
- **Data privacy.** Patient data is personal data. The Digital Personal Data Protection Act, 2023 places obligations on organisations that collect it, so a hospital tool should collect only what it needs.
- **Fairness of automated decisions.** A scoring system must be transparent and must support, not replace, clinical judgement.
- **Web application security.** The OWASP Top Ten (2021) lists injection, cross-site scripting and broken access control among the most common web risks, so even a small application must handle them deliberately.

## 1.2. Identification of Problem

The broad problem addressed by this project is that hospital beds are allocated in a manual and subjective way. In particular:

- waiting patients are recorded in different places, so staff do not know the true queue or the number of free beds;
- the choice of the next patient depends on the person on duty, so similar patients may be treated differently;
- a patient who waits quietly may be overlooked while another patient is admitted first;
- when the ICU is full there is no consistent rule for using Emergency or General beds;
- there is no history of why a bed was given to a particular patient; and

- correcting or removing a wrongly entered record means changing several lists.

The problem to be solved, therefore, is how to give a hospital a dependable, shared and explainable way to register patients, rank them fairly and assign beds, using only open web technologies that a small team can build and maintain.

## 1.3. Identification of Tasks

The work was divided into nine tasks: requirement analysis and literature study (T1), feature evaluation and constraints (T2), database and API design (T3), backend implementation (T4), priority and allocation logic (T5), frontend implementation (T6), frontend and backend integration (T7), testing and validation (T8) and documentation (T9). These tasks follow the framework of this report: analysis in Chapter 1, design in Chapter 2, implementation and testing in Chapter 3 and the outcome in Chapter 4.

## 1.4. Timeline

The project was planned over eight weeks. Figure 1.1 shows the Gantt chart used to schedule and track the tasks. Backend and frontend work overlapped, and the frontend was started as soon as the first API routes returned data. Report preparation ran in parallel with testing. The tasks were shared among the team members as shown in Table 1.1. Both members took part in design discussions, code review and testing.

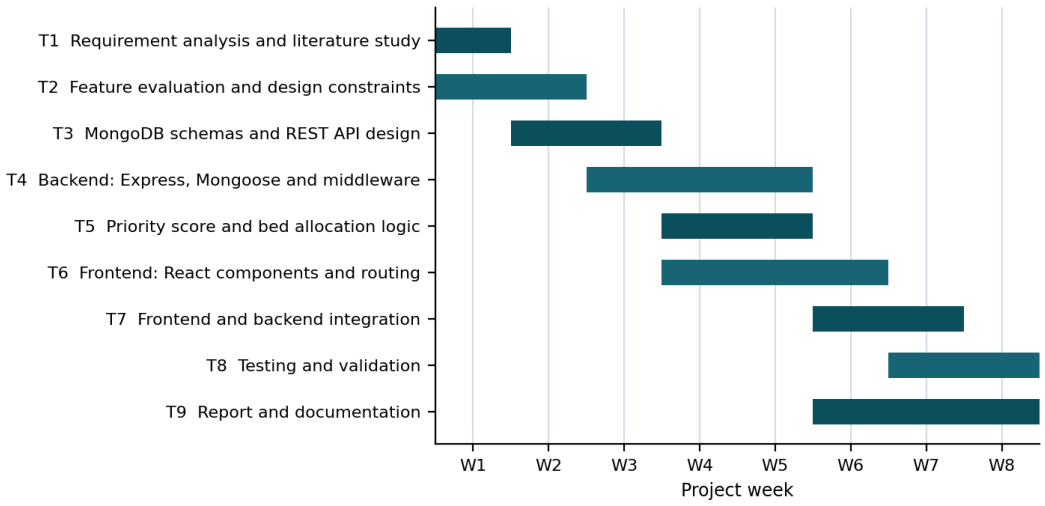



**Figure 1.1: Gantt chart of the project timeline**

**Table 1.1: Team members and primary responsibilities**

| Team member | UID | Primary responsibilities |
|---|---|---|
| Ishita Srivastava | 24BCS10823 | Mongoose models, Express routes and middleware, priority and allocation logic, API testing |
| Priyanshi Gupta | 24BCS10819 | React components and routing, forms and styling, output screens, report compilation |

## 1.5. Organization of the Report

Chapter 1 identifies the client, need, problem, tasks and timeline. Chapter 2 presents the features and constraints, compares two alternative designs, justifies the final design and gives the implementation plan. Chapter 3 describes the implementation, the output screens, testing and data validation. Chapter 4 gives the conclusion, the deviations from the expected results and the future work. References, an appendix, the user manual and the achievements follow.

# CHAPTER 2.
# DESIGN FLOW/PROCESS

## 2.1. Concept Generation, Evaluation & Selection of Specifications/Features

Concept generation began by studying how a hospital admissions desk works and listing every function that staff might reasonably want. Each candidate feature was rated for priority (essential, desirable or optional) and given a decision (include, defer or reject). The evaluation is summarised in Table 2.1.

**Table 2.1: Evaluation of candidate features**

| Feature | Priority | Decision | Reason |
|---|---|---|---|
| Register, edit and delete patients | Essential | Include | Basis of every other function |
| Priority score from severity, waiting time, distance and age | Essential | Include | Objective and explainable ranking |
| Automatic bed allocation by ward preference, with fallback | Essential | Include | Core purpose of the system |
| Release a patient and free the bed | Essential | Include | Beds must become available again |
| Ward occupancy dashboard | Essential | Include | Gives a shared picture of free beds |
| Waiting queue view | Essential | Include | Shows who is next and why |
| Server-side validation with clear messages | Essential | Include | Protects data quality |
| Allocation log | Desirable | Include | Auditable history of decisions |
| Search, filter, sort and pagination | Desirable | Include | Needed as the patient list grows |
| Login and user roles (bcrypt and JWT) | Desirable | Defer | Module written in the backend but not enforced in the delivered version |
| Doctor and nurse scheduling | Optional | Defer | Beyond the scope of bed management |
| Live push updates (WebSockets) | Optional | Defer | Refreshing every 5 seconds is sufficient |
| Billing, pharmacy and electronic health records | Optional | Reject | A different problem domain |
| SMS or e-mail alerts | Optional | Reject | Needs an external paid service |

## 2.2. Design Constraints

Table 2.2 summarises the constraints that were considered while designing the system and how each was addressed.

**Table 2.2: Design constraints and how they were addressed**

| Constraint | Consideration in the design |
|---|---|
| Regulations | Patient data is sensitive personal data. The design avoids identity numbers, contact details and full medical records, and the demonstration uses fictional patients, in line with the data-minimisation idea of the Digital Personal Data Protection Act, 2023. A real deployment would also need consent, retention and access rules. |
| Economic | Only free and open-source software is used (Node.js, React, Express, MongoDB Community Edition, VS Code). The hardware requirement is one ordinary computer, so the cost is close to zero. |
| Environmental | The application is lightweight and runs on a single low-power machine. It replaces paper forms and printed lists, and refreshing every 5 seconds keeps network and processor use small. |
| Health | The system supports clinical staff but does not diagnose. Priority scores are advisory and are shown with their values so that staff can overrule them. Severity is shown with a name as well as a colour. |
| Manufacturability (Buildability) | The project runs with two commands per application (npm install and npm start or npm run dev), has a standard folder layout and lists all dependencies in package.json, so another developer can rebuild it on any computer with Node.js. |
| Safety / Security | Every write passes Mongoose schema validation, search text is escaped before it is used as a regular expression, only named fields are read from request bodies, and React escapes all displayed text. Access control is not enforced in the delivered version, so it must be used on a trusted network. |
| Professional | The code follows a layered structure with consistent naming, REST conventions, HTTP status codes and comments that link each module to a course experiment. A README explains set-up. |
| Ethical | The scoring formula is open and documented. It does not use gender, religion, caste, income or insurance. Age is used only to protect the very young and the elderly. Final decisions stay with clinicians. |
| Social | The interface uses plain English and a small number of screens so that staff with limited technical training can use it. The aim is that the most urgent patients are treated first and that waiting is handled fairly. |
| Political | No political or legal restriction applies to the technology chosen. All components are openly licensed and not dependent on a single vendor. |

## 2.3. Analysis and Feature Finalization Subject to Constraints

The feature list of Table 2.1 was reviewed against the constraints of Table 2.2. Billing, notifications and electronic health records were rejected because they need paid services and much more sensitive data. Doctor scheduling and live push updates were deferred. Login and user roles were developed as a backend module (registration and login with bcrypt-hashed passwords and JWT tokens), but the delivered version does not enforce them, so authentication is treated as a deferred feature and is listed under future work in Chapter 4.

**Final feature set:** register, edit and delete patients; a priority score and a ranked waiting queue; automatic bed allocation (next patient or all waiting patients) with ward fallback; release of patients; a ward occupancy dashboard that refreshes automatically; an allocation log; search by name, filtering by severity and status, sorting and pagination; and server-side validation with clear error messages.

### 2.3.1. Priority score

The priority score decides the order of the waiting queue. For a patient with severity S (1 to 5), waiting time W in minutes, distance D in kilometres and age bonus A, the score P is:

$$P = 40 \times S + \min(0.5 \times W, 30) + \max(0, 20 - 0.5 \times D) + A$$

where A is 10 when the patient is aged 5 or under, or 65 or over, and 0 otherwise. The maximum possible score is 260. Severity carries the largest weight. Waiting time, distance and age adjust the order among patients of similar urgency and make sure that a long wait is not ignored: waiting adds up to 30 points, so a patient who has waited a long time can overtake a newly arrived patient of slightly higher severity.

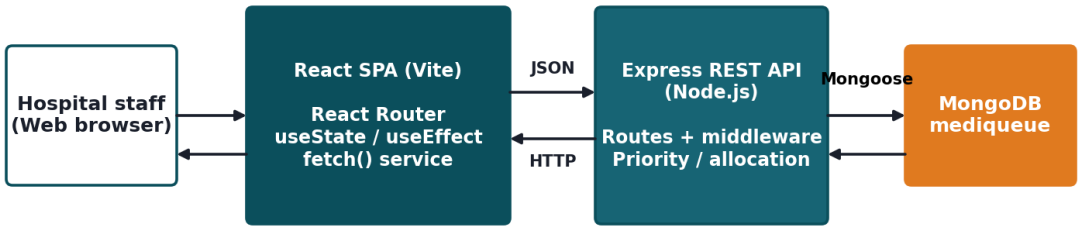
Each patient is then offered wards in an order that depends on severity: severity 4 or 5 tries ICU, then Emergency, then General; severity 3 tries Emergency, then ICU, then General; severity 1 or 2 tries General, then Emergency, then ICU. The hospital has 30 beds: 5 in ICU, 10 in Emergency and 15 in General.

## 2.4. Design Flow

Two alternative designs were prepared to build the final feature set. Both use MongoDB for storage and Node.js on the server; they differ in how the user interface is produced.

### 2.4.1. Design A: React single-page application + Express REST API + MongoDB

In Design A the browser loads a React single-page application built with Vite. React Router changes screens without reloading the page, hooks hold state and fetch data, and a small fetch service sends JSON requests to an Express REST API. The API validates input, runs the allocation logic and uses Mongoose to read and write MongoDB. The two applications are developed and started separately. The block diagram is shown in Figure 2.1.

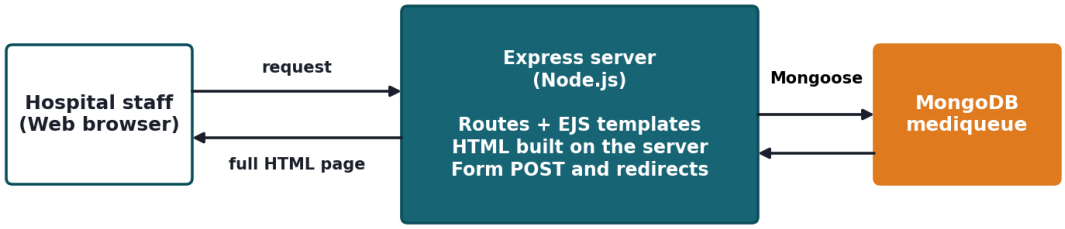



*Design A: single-page application with a separate REST API*

**Figure 2.1: Block diagram of Design A (React SPA + Express REST API)**

### 2.4.2. Design B: Server-rendered Express application + EJS + MongoDB

In Design B a single Express application builds every page on the server using an EJS template engine and sends complete HTML to the browser. Actions such as registering a patient are HTML form posts followed by a redirect. There is one application to run, but each action reloads the whole page, the templates introduce a technology that is not part of the course, and the server cannot easily be reused by another client. The block diagram is shown in Figure 2.2.




*Design B: server-rendered Express application with EJS views*

**Figure 2.2: Block diagram of Design B (server-rendered Express + EJS)**

## 2.5. Design Selection

The two designs were compared on six criteria. Each criterion has a weight from 1 to 5 and each design was scored from 1 (poor) to 5 (excellent). The weighted comparison is given in Table 2.3.

**Table 2.3: Comparison of Design A and Design B**

| Criterion | Weight | Design A score | Design B score |
|---|---|---|---|
| Alignment with the course outcomes (CO1 to CO5) | 5 | 5 | 2 |
| Responsive interface without page reloads | 4 | 5 | 2 |
| Separation of concerns and maintainability | 4 | 4 | 3 |
| Reuse of the API by other clients | 3 | 5 | 2 |
| Ease of automated API testing | 3 | 5 | 3 |
| Development effort | 3 | 3 | 4 |
| **Weighted total (maximum 110)** | | **100** | **57** |

**Best design: Design A.** It scores 100 out of 110 against 57 for Design B. It matches the course outcomes directly (React components, hooks and routing in CO1; REST APIs in CO2; Mongoose in CO3; integration in CO4), gives a faster interface because screens update without reloading, and keeps the user interface and the business logic apart so that each can be tested and changed on its own. Its disadvantage is that two applications must be started, which is a small cost compared with the benefits. Design A was therefore selected and implemented.

## 2.6. Implementation Plan/Methodology

The implementation followed the sequence: (1) create the Vite React project and the Express project; (2) define the Mongoose models; (3) write the middleware; (4) write the routes; (5) write the priority and allocation modules; (6) build the React pages and components; (7) connect the frontend to the API with fetch; (8) test every route and rule; and (9) prepare the documentation. The parts are described below.

### 2.6.1. Layered architecture

The system is organised in seven layers (Figure 2.3). Each layer talks only to the layer next to it, so a change such as a new scoring rule touches only the business logic layer.

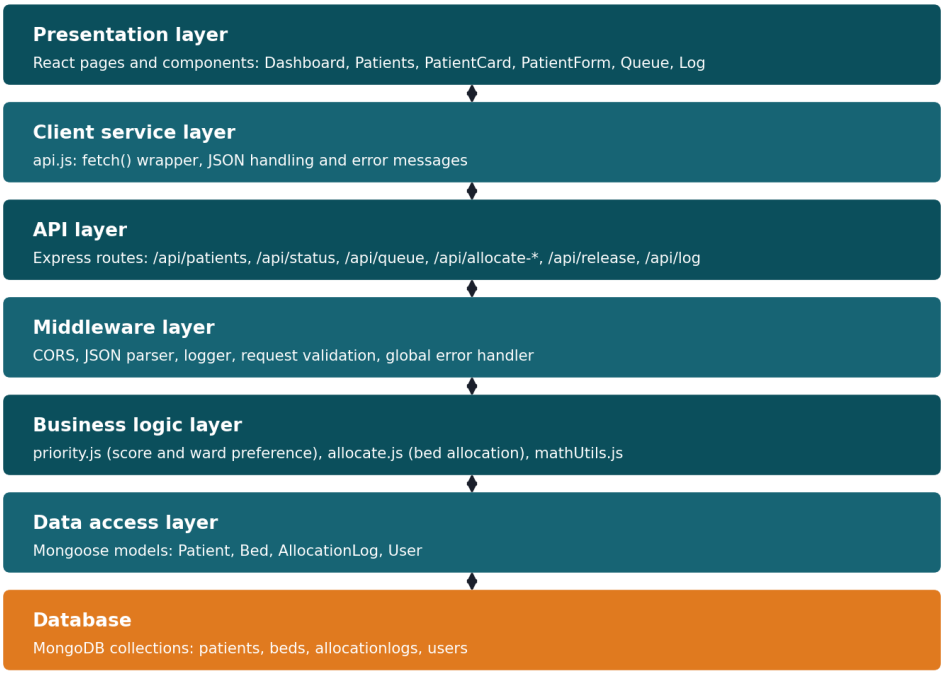



**Figure 2.3: Layered architecture of MediQueue**

### 2.6.2. Database design

MongoDB holds four collections (Figure 2.4). *Patient* stores the details entered by staff, a status (waiting, admitted or discharged) and a reference to the assigned bed. *Bed* stores a unique bed identifier, the ward, whether it is free and a reference to the patient. *AllocationLog* records each allocation with the score and waiting time at that moment. *User* holds the accounts of the authentication module. Field-level rules such as required, minimum length, minimum and maximum values and an enumerated list of statuses are written in the Mongoose schemas.

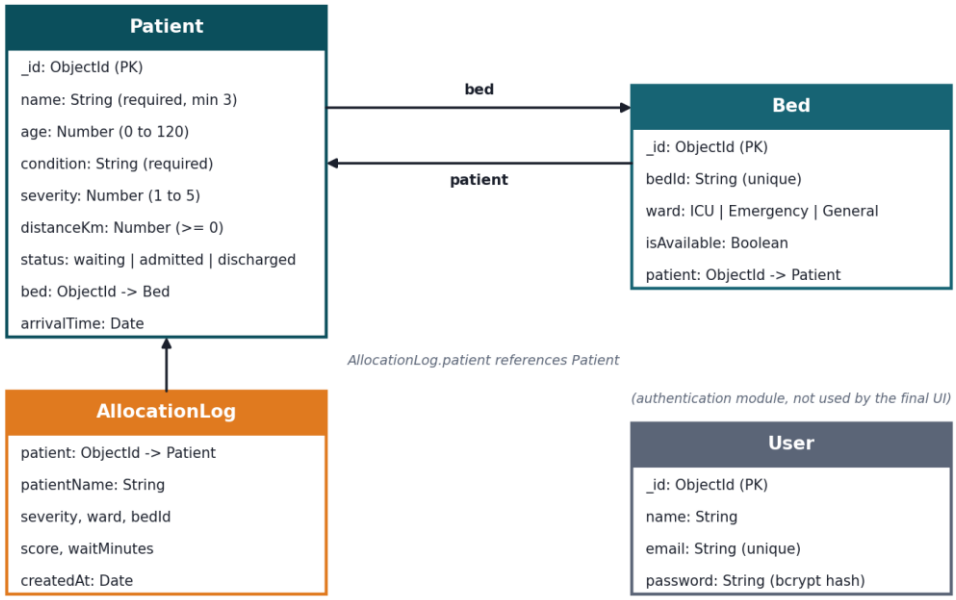



**Figure 2.4: Database schema of the collections**

### 2.6.3. Allocation algorithm

The allocation algorithm is shown as a flowchart in Figure 2.5 and has the following steps:

1. Read all patients whose status is waiting. If there are none, return “empty”.
2. Compute the priority score of each patient, sort in descending order and take the first patient.
3. Get the ward preference list for the patient’s severity.
4. For each ward in that order, look for the free bed with the lowest identifier.
5. If a bed is found, mark it occupied, set the patient to admitted, link the two, write an AllocationLog entry and return “allocated”.
6. If no ward has a free bed, return “no_bed”; the patient stays in the queue.

“Allocate all” repeats the algorithm until the result is no longer “allocated”. Releasing a patient frees the bed and sets the patient to discharged.

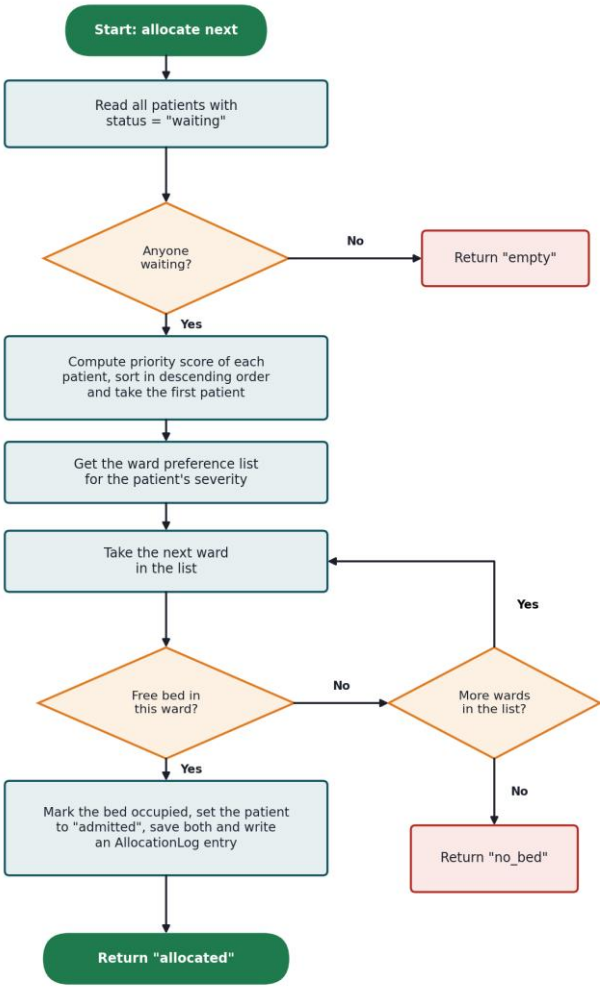



**Figure 2.5: Flowchart of the bed allocation algorithm**

### 2.6.4. Request handling and validation

Every request passes through the same chain (Figure 2.6). Cross-origin access and JSON parsing come first, then a logger records the method, URL and time. Unknown routes receive a 404 response. For POST and PUT requests a validation middleware rejects requests with missing fields before any database call. Mongoose then validates the values, and any error thrown is converted by a single global error handler into a consistent JSON response.

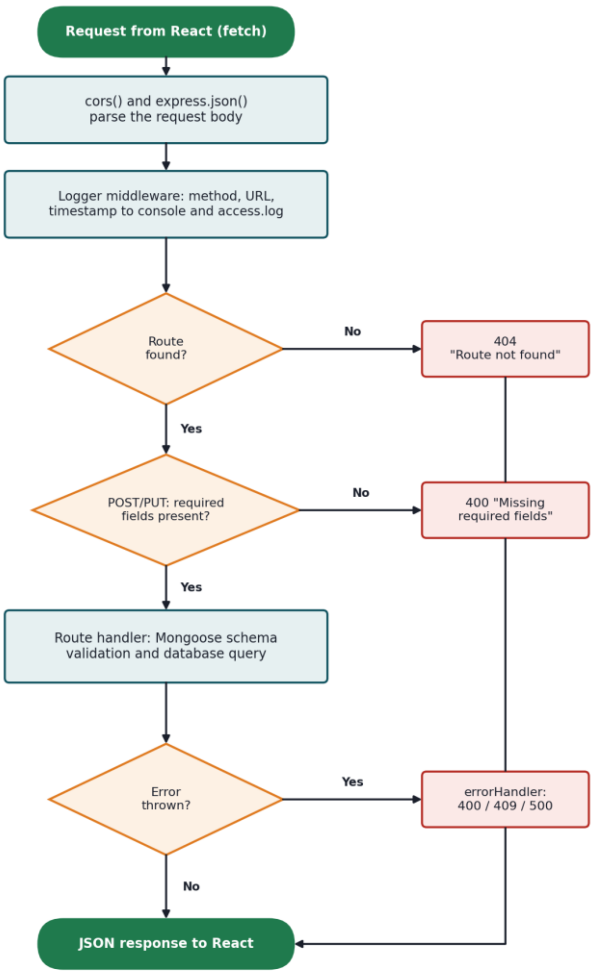



**Figure 2.6: Flowchart of request handling and validation**

### 2.6.5. REST API

The endpoints of the API are listed in Table 2.4. All responses are JSON and use the HTTP status codes 200, 201, 400, 404, 409 and 500.

**Table 2.4: REST API endpoints**

| Method | Route | Purpose | Success code |
|---|---|---|---|
| POST | /api/register | Create a user (password hashed with bcrypt) | 201 |
| POST | /api/login | Check credentials and return a JWT | 200 |
| GET | /api/profile | Return the current user | 200 |
| GET | /api/patients | List patients; query: search, severity, status, sort, page, limit | 200 |
| GET | /api/patients/:id | Return one patient with bed details | 200 |
| POST | /api/patients | Register a patient (status starts as waiting) | 201 |
| PUT | /api/patients/:id | Edit name, age, condition, severity or distance | 200 |
| DELETE | /api/patients/:id | Delete a patient and free any bed | 200 |

| Method | Route | Purpose | Success code |
|---|---|---|---|
| GET | /api/status | Beds per ward, free beds and patients waiting | 200 |
| GET | /api/queue | Waiting patients ranked by priority score | 200 |
| POST | /api/allocate-next | Place the highest-priority patient in a bed | 200 |
| POST | /api/allocate-all | Place waiting patients until no bed is left | 200 |
| POST | /api/release/:id | Release an admitted patient and free the bed | 200 |
| GET | /api/log | Latest 50 allocations, newest first | 200 |
| POST | /api/demo | Add 10 sample patients to the queue | 201 |
| POST | /api/reset | Delete all patients and logs; free all beds | 200 |

# CHAPTER 3.
# RESULTS ANALYSIS AND VALIDATION

## 3.1. Implementation of Solution

### 3.1.1. Tools and technologies

The solution was implemented with the tools in Table 3.1. All of them are free and open source. The technologies at the core of the project are the ones taught in the Fullstack Development – I syllabus; nodemon, dotenv and the editor tools are supporting tools.

**Table 3.1: Tools and technologies used**

| Tool | Version | Purpose |
|---|---|---|
| Node.js | 18 or later | JavaScript runtime for the backend |
| Express | 4.19 | Routing, middleware and REST API |
| MongoDB | Community / Atlas | Document database |
| Mongoose | 8.5 | Schemas, validation, queries and populate |
| React | 18.3 | Component-based user interface |
| Vite | 5.4 | Development server and build tool |
| React Router | 6.26 | Client-side routing for the single-page application |
| bcryptjs, jsonwebtoken | 2.4, 9.0 | Password hashing and tokens of the authentication module |
| cors, dotenv | 2.8, 16.4 | Cross-origin access and environment settings |
| nodemon | 3.1 | Automatic restart of the server during development |
| VS Code, MongoDB Compass | – | Editor and database viewer |

### 3.1.2. Project directory structure

The project is two independent applications in one folder (Figure 3.1). The backend is about 550 lines of JavaScript and the frontend about 640 lines of JavaScript and JSX, excluding the stylesheet.

```
mediqueue/
|-- README.md
|-- backend/
|   |-- server.js            Express entry point
|   |-- seed.js              optional sample data
|   |-- package.json         dependencies and scripts
|   |-- .env / .env.example  port, MongoDB URI, JWT secret
|   |-- config/              db.js, beds.js
|   |-- models/              User, Patient, Bed, AllocationLog
|   |-- routes/              auth, patients, hospital
|   |-- middleware/          logger, validate, verifyToken, errorHandler
|   `-- utils/               mathUtils, priority, allocate
`-- frontend/
    |-- index.html, package.json, vite.config.js
    `-- src/
        |-- main.jsx, App.jsx, api.js, severity.js, styles.css
        |-- components/      Layout, Sidebar, PatientCard,
        |                    PatientForm, Spinner
        `-- pages/           Dashboard, Patients, PatientDetail,
                             AddPatient, Queue, Log, NotFound
```

**Figure 3.1: Project directory structure**

### 3.1.3. Backend modules

`models/Patient.js` defines the Patient schema with required fields, a minimum name length of 3, an age range of 0 to 120, a severity range of 1 to 5, a non-negative distance, an enumerated status and a reference to Bed. `Bed.js` and `AllocationLog.js` define the other collections. `config/beds.js` creates the 30 beds (ICU-B01 to ICU-B05, Emergency-B01 to Emergency-B10 and General-B01 to General-B15) and adds any that are missing each time the server starts.

`utils/priority.js` holds the scoring rule and the ward preference as a custom Node module, built on the small `mathUtils` module (add, subtract, multiply, divide). `utils/allocate.js` implements the algorithm of Figure 2.5. The core of the scoring function is:

```
function priorityScore(patient, now = Date.now()) {
  const waitingMinutes = subtract(now, new Date(patient.arrivalTime).getTime()) / 60000;
  const severityScore = multiply(patient.severity, 40);
  const waitingScore = Math.min(waitingMinutes * 0.5, 30);
  const distanceScore = Math.max(0, 20 - (patient.distanceKm || 0) * 0.5);
  const ageScore = patient.age >= 65 || patient.age <= 5 ? 10 : 0;
  return add(add(severityScore, waitingScore), add(distanceScore, ageScore));
}
```

The middleware folder contains the logger (method, URL and time written to the console and appended to `logs/access.log` with the `fs` module), the validation middleware, the global error handler and the token-verification middleware. The error handler maps a Mongoose `ValidationError` to 400, a `CastError` (malformed id) to 400, a duplicate-key error to 409 and anything else to 500. In the delivered version the token-verification middleware passes every request, because the login screens were removed.

`routes/patients.js` implements the patient CRUD operations. The list route builds a query from the search text, severity and status filters, then applies `populate`, `sort`, `skip` and `limit`. User text is escaped before it becomes a regular expression:

```
const safe = search.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
filter.name = { $regex: safe, $options: 'i' };
```

`routes/hospital.js` provides the status, queue, allocation, release, log, sample-data and reset routes.

### 3.1.4. Frontend modules

`App.jsx` declares the routes: Dashboard, Patients, Patient detail (`/patients/:id`), Add patient, Waiting queue, Allocation log and a 404 page, all inside a `Layout` component that shows the `Sidebar` and an `Outlet`. The Patients page keeps the list, filters, page number and loading and error state in `useState` and refetches inside `useEffect` whenever a filter, the page or a reload counter changes. The Dashboard and Queue pages refresh every 5 seconds with `setInterval` and clear the timer when the page is left. `PatientCard` receives its data through props, `PatientForm` is a controlled form used for both adding and editing, and `Spinner` is shown while data loads. Empty lists, errors and the edit window use conditional rendering. All requests go through `api.js`, which wraps `fetch`.

## 3.2. Output Screens

The main screens are shown in Figures 3.2 to 3.6, rendered from the project's actual layout and stylesheet with sample data. The data was produced by running the project's own allocation code on ten sample patients: six were placed in beds and four were left waiting. The dashboard shows free, occupied and waiting counts, the allocation buttons, the result of the last allocation and the occupancy of each ward.

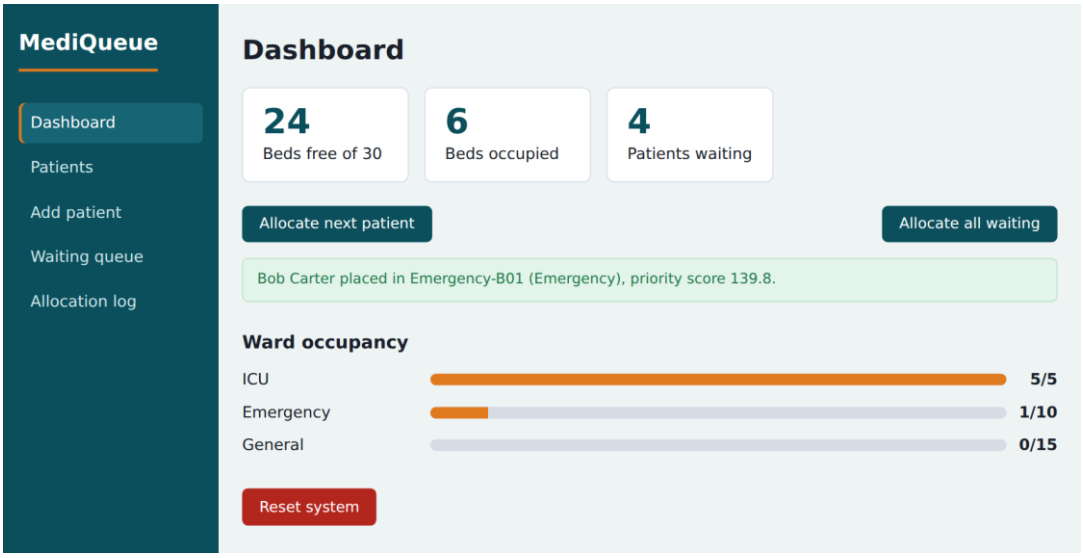



**Figure 3.2: Dashboard with bed counts, actions and ward occupancy**

The waiting queue lists patients in descending score order, with severity, time waited and score. The highest-scoring patient (rank 1) is the one that "Allocate next patient" will place.

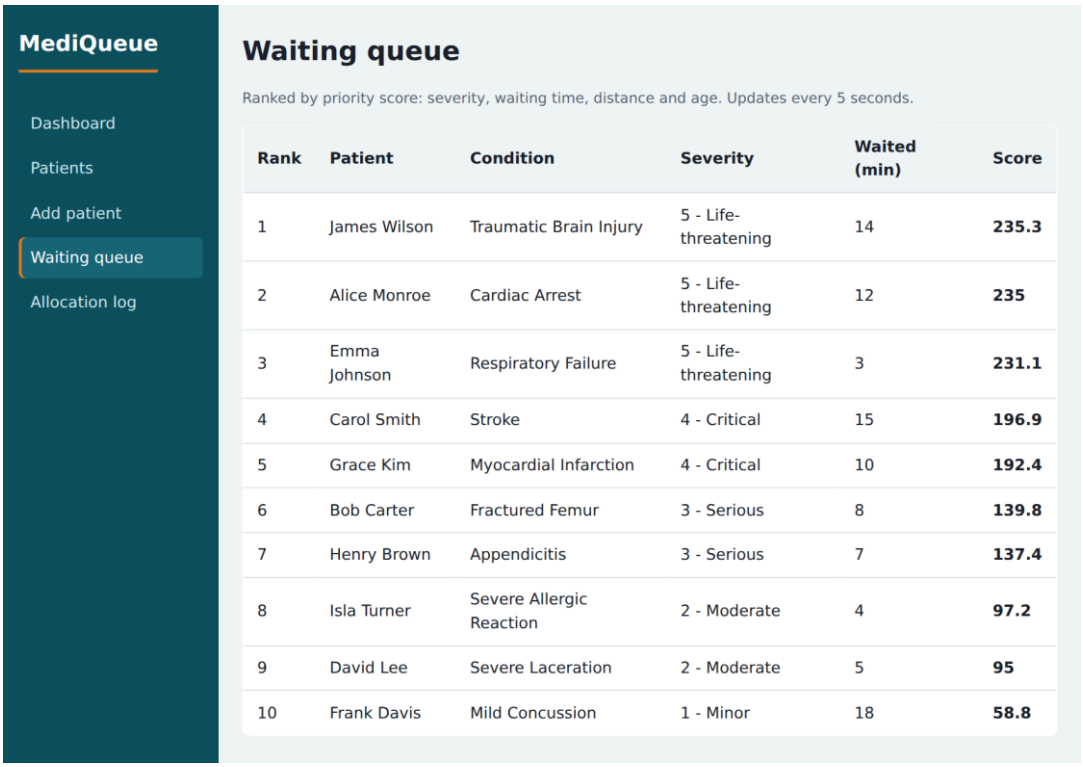



**Figure 3.3: Waiting queue ranked by priority score**

The patient list shows each patient with severity, status and bed. Admitted patients have a Release button. The list can be searched, filtered by severity and status, sorted and paged.

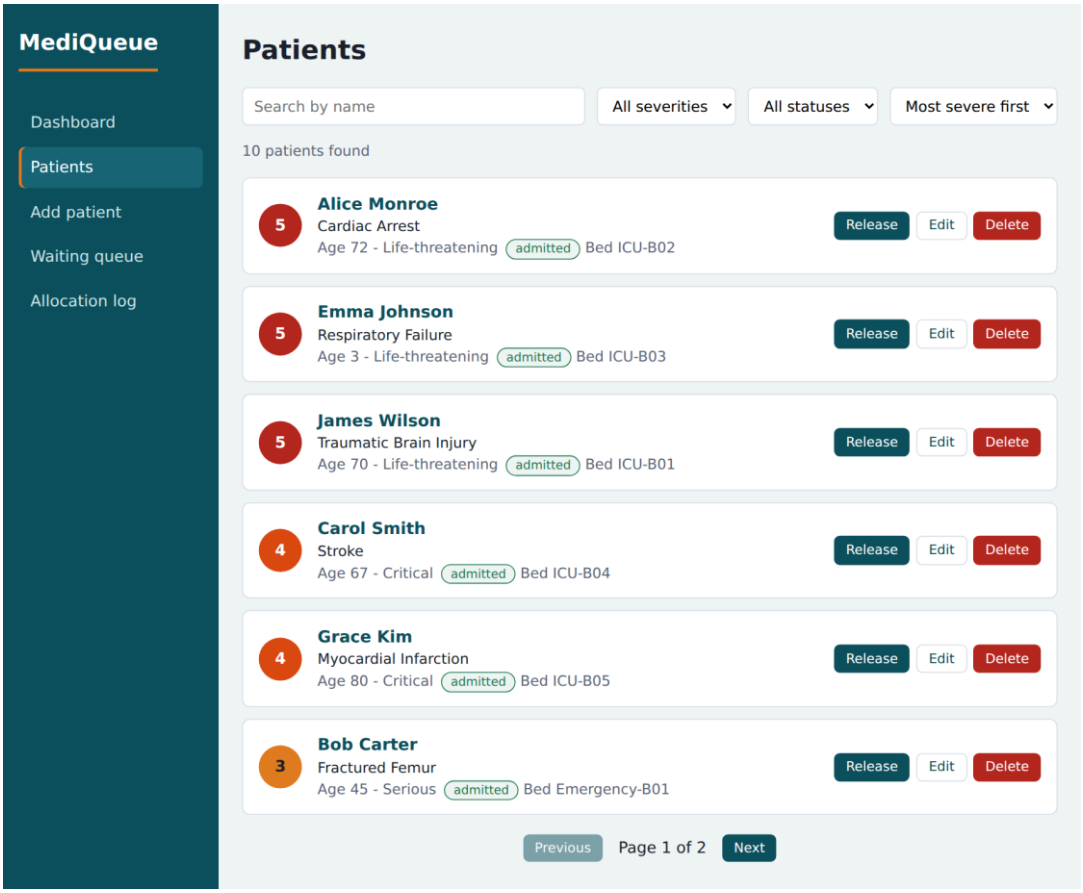



**Figure 3.4: Patient list with search, filters, paging and actions**

When invalid data is submitted, the message from the server is shown above the form and the entered values are kept. Here the name is only two characters long.

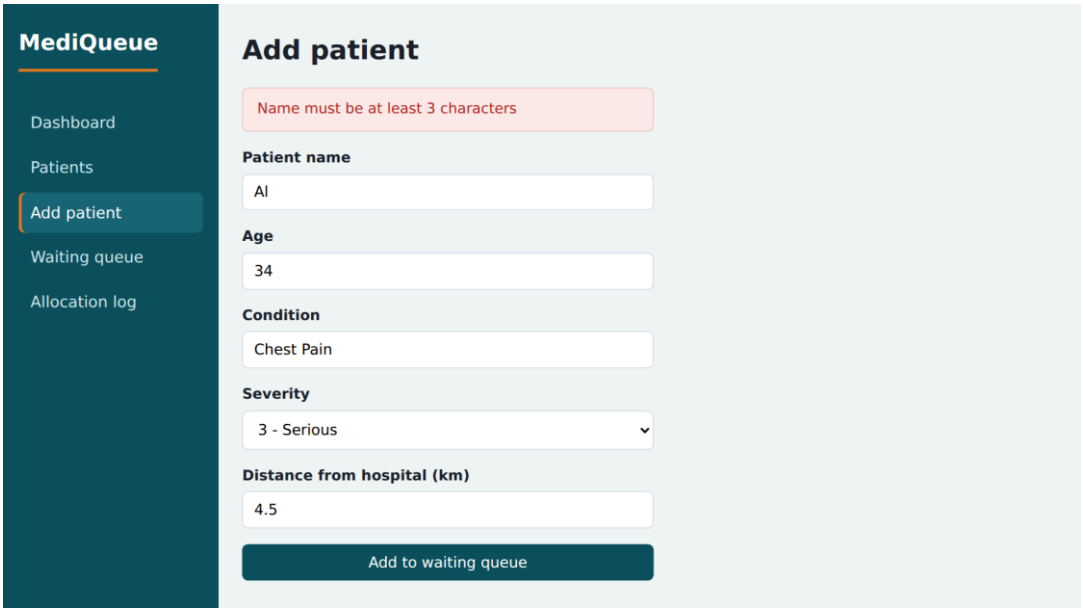



**Figure 3.5: Add patient form showing a validation message**

The allocation log records each placement with the ward, bed, score and time waited. The newest entry is at the top.

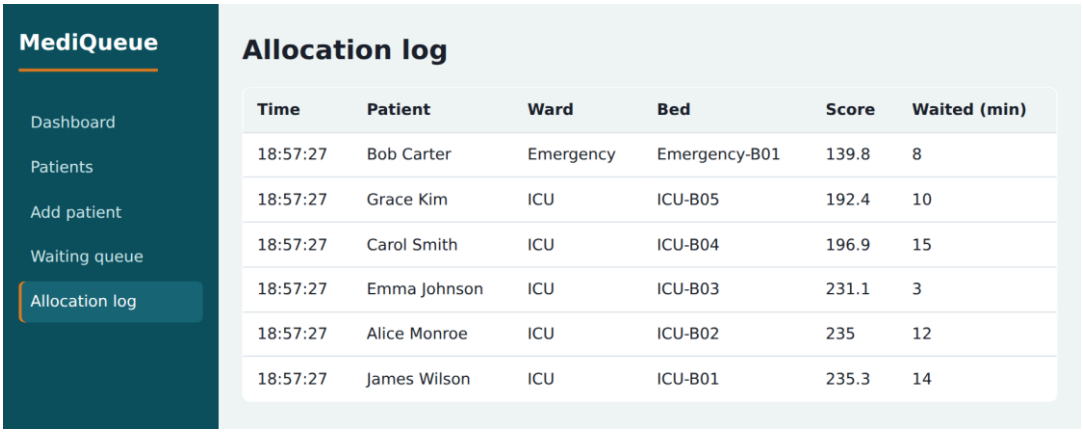



**Figure 3.6: Allocation log**

## 3.3. Testing, Characterization and Data Validation

Testing was carried out with an automated Node.js script that starts the project's own Express application, with its real routes, middleware, Mongoose schemas and allocation code, and sends HTTP requests to it. MongoDB was not available in the test environment, so the database layer was replaced by an in-memory store; schema validation, which runs before any database call, therefore used the real Mongoose code. In total 44 test cases were run: 17 functional, 16 validation and negative, 7 priority-score and 4 error-handling cases. All 44 passed. The results are in Tables 3.2 to 3.4.

**Table 3.2: Functional test cases**

| ID | Test case | Input / action | Expected result | Observed result | Status |
|---|---|---|---|---|---|
| F1 | API root responds | GET / | 200 and message "MediQueue API is running" | 200 "MediQueue API is running" | Pass |
| F2 | Beds created on first start | ensureBeds() then GET /api/status | ICU 5, Emergency 10, General 15 (30 beds) | ICU 5, Emergency 10, General 15 (30 beds) | Pass |
| F3 | Bed creation is repeatable | Run ensureBeds() a second time | No duplicate beds are added | 30 beds before, 30 after | Pass |
| F4 | Register a patient | POST /api/patients with valid data | 201; status "waiting" | 201; status "waiting" | Pass |
| F5 | Load sample patients | POST /api/demo | 201; 10 patients loaded | 201; 10 loaded | Pass |
| F6 | Waiting queue ranked by score | GET /api/queue | 10 patients in descending score order; rank 1 has severity 5 | 10 patients; descending=true; rank 1 = Emma Johnson (severity 5, score 229.6) | Pass |
| F7 | Allocate next patient | POST /api/allocate-next | Highest-priority patient placed in an ICU bed | allocated: Emma Johnson -> ICU-B01 (ICU), score 229.6 | Pass |
| F8 | Allocate all waiting patients | POST /api/allocate-all | The 9 remaining patients are placed; none waiting | 9 placed; waiting 0; occupied ICU 5, Emergency 2, General 3 | Pass |

| ID | Test case | Input / action | Expected result | Observed result | Status |
|---|---|---|---|---|---|
| F9 | Allocation log | GET /api/log | 10 entries, newest first | 10 entries; first = Frank Davis | Pass |
| F10 | Release a patient | POST /api/release/:id | 200; patient "discharged"; bed freed (occupied 10 -> 9) | 200; "discharged"; occupied 9 | Pass |
| F11 | Search by name (regex) | GET /api/patients?search=ali | Only Alice Monroe | Alice Monroe | Pass |
| F12 | Filter by severity | GET /api/patients?severity=5 | 3 patients (all severity 5) | 3 patients | Pass |
| F13 | Sort by age | GET /api/patients?sort=age_desc | Oldest first (Grace Kim, 80) | Grace Kim (80) | Pass |
| F14 | Pagination | GET /api/patients?page=2&limit=6 | Page 2 of 2 with the remaining 4 patients | page 2 of 2, 4 patients, total 10 | Pass |
| F15 | Edit a patient | PUT /api/patients/:id | 200 and the new condition is saved | 200; condition "Updated Condition" | Pass |
| F16 | Delete an admitted patient | DELETE /api/patients/:id | 200; the bed is freed | 200; occupied 9 -> 8 | Pass |
| F17 | Unknown patient id | GET /api/patients/<unknown id> | 404 "Patient not found" | 404 "Patient not found" | Pass |

**Table 3.3: Validation, boundary and negative test cases**

| ID | Test case | Input / action | Expected result | Observed result | Status |
|---|---|---|---|---|---|
| V1 | Missing required fields | POST /api/patients with {} | 400 listing the missing fields | 400 "Missing required fields: name, age, condition, severity" | Pass |
| V2 | Name shorter than 3 characters | POST with name "Al" | 400 "Name must be at least 3 characters" | 400 "Name must be at least 3 characters" | Pass |
| V3 | Age above 120 | POST with age 130 | 400 "Age must be 120 or less" | 400 "Age must be 120 or less" | Pass |
| V4 | Negative age | POST with age -1 | 400 "Age cannot be negative" | 400 "Age cannot be negative" | Pass |
| V5 | Severity above 5 | POST with severity 6 | 400 "Severity must be 1 to 5" | 400 "Severity must be 1 to 5" | Pass |
| V6 | Severity 0 (boundary) | POST with severity 0 | 400 "Severity must be 1 to 5" | 400 "Severity must be 1 to 5" | Pass |
| V7 | Negative distance | POST with distanceKm -5 | 400 "Distance cannot be negative" | 400 "Distance cannot be negative" | Pass |
| V8 | Boundary values accepted | POST severity 1 and severity 5 with age 0 | 201 for both | 201 and 201 | Pass |
| V9 | Script text in a name | POST with name "<script>alert(1)</script>" | Stored as plain text (React escapes it on display) | 201; stored "<script>alert(1)</script>" | Pass |
| V10 | Regex characters in search | GET /api/patients?search=(.*[ | 200 with no match; no server error | 200; 0 matches | Pass |

| ID | Test case | Input / action | Expected result | Observed result | Status |
|---|---|---|---|---|---|
| V11 | Mass-assignment attempt | POST with extra fields status and bed | Extra fields ignored; status stays "waiting" | status "waiting", bed null | Pass |
| V12 | Unknown route | GET /api/nothing-here | 404 JSON "Route not found" | 404 "Route not found" | Pass |
| V13 | Allocate with nobody waiting | POST /api/allocate-next on an empty queue | status "empty" | status "empty" | Pass |
| V14 | Release a patient who is not in a bed | POST /api/release/<unknown id> | 404 | 404 "Patient not found or not in a bed" | Pass |
| V15 | Ward fallback when ICU is full | Six severity-5 patients, then allocate all | ICU takes 5; the sixth goes to Emergency | ICU 5, Emergency 1, General 0 | Pass |
| V16 | No bed available | 31 minor patients, 30 beds, then allocate next | Last patient stays waiting; status "no_bed" | status "no_bed"; occupied 30; waiting 1 | Pass |

**Table 3.4: Priority score and error-handling test cases**

| ID | Test case | Input / action | Expected result | Observed result | Status |
|---|---|---|---|---|---|
| P1 | Severity weight | Score for severity 3 and severity 4 (other inputs equal) | Difference of exactly 40 | 140 and 180; difference 40 | Pass |
| P2 | Waiting time adds 0.5 per minute | 20 minutes of waiting | 10 extra points | 10 extra points | Pass |
| P3 | Waiting bonus is capped | 1000 minutes of waiting | At most 30 extra points | 30 extra points | Pass |
| P4 | Distance bonus never negative | Distance 0 km and 100 km | 20 points at 0 km; floor of 0 far away | 20 point swing | Pass |
| P5 | Age bonus for the very young and the elderly | Ages 3, 40 and 65 | +10 for ages 5 or under and 65 or over | age 3: +10, age 65: +10 | Pass |
| P6 | Ward preference order | Severity 4, 3 and 1 | ICU>Emergency>General; Emergency>ICU>General; General>Emergency>ICU | ICU>Emergency>General; Emergency>ICU>General; General>Emergency>ICU | Pass |
| P7 | Longer wait moves a patient up | Two identical patients, one waiting 60 minutes | The patient who waited longer is rank 1 | rank 1 = Waited Long (167.5); rank 2 = Just Arrived (137.5) | Pass |
| E1 | Error handler: validation error | errorHandler(ValidationError) | 400 with the field message | 400 "Age cannot be negative" | Pass |
| E2 | Error handler: malformed id | errorHandler(CastError) | 400 "Invalid ID format" | 400 "Invalid ID format" | Pass |
| E3 | Error handler: duplicate key | errorHandler(code 11000) | 409 | 409 | Pass |
| E4 | Error handler: unexpected error | errorHandler(Error) | 500 JSON with success:false | 500 success=false | Pass |

### 3.3.1. Performance assessment

The priority computation was timed by sorting arrays of generated patients with the real scoring function (Table 3.5). Even for 10,000 patients the sort took about 50 milliseconds, far more than a hospital queue will ever hold. These figures measure only the scoring and sorting in Node.js; response times with a real MongoDB server and a network were not measured.

**Table 3.5: Performance of the priority computation**

| Number of patients | Sort time (ms) | Time per patient (ms) |
|---|---|---|
| 100 | 0.9 | 0.0090 |
| 1000 | 12 | 0.0120 |
| 10000 | 50 | 0.0050 |

### 3.3.2. Security analysis and interpretation

**Table 3.6: Security analysis**

| Threat | Mitigation | Residual risk |
|---|---|---|
| Query injection | Mongoose casts values to the schema types; search text is escaped before use as a regular expression (test V10) | Low |
| Cross-site scripting | React escapes displayed text; the source contains no dangerouslySetInnerHTML (test V9) | Low |
| Invalid data | Schema validation and validation middleware (tests V1 to V8) | Low |
| Mass assignment | Routes copy only the five allowed fields (test V11) | Low |
| Unauthenticated access | Authentication module exists but is not enforced; anyone who can reach port 5000 can read and change data | High if exposed beyond a trusted network |
| Open cross-origin access | cors() accepts every origin | Medium |
| Secrets in configuration | .env is excluded from version control and an example file is provided; the placeholder JWT secret must be changed | Medium |
| Destructive actions | Delete and Reset need only a confirmation dialog | Medium |

All functional, validation, priority and error-handling cases behaved as expected. The layered design made faults easy to locate: a wrong value was caught by the schema, a missing field by the validation middleware and a rule error by the scoring module. Every requirement of Chapter 1 was met in the delivered version: a shared record of patients and beds, an explainable ranking, automatic ward matching with fallback, an allocation history, search and correction of records, and a simple interface. The main limitation is that access control is not enforced; the security table shows that this is the highest remaining risk.

# CHAPTER 4.
# CONCLUSION AND FUTURE WORK

## 4.1. Conclusion

The aim of the project was to replace manual bed allocation with a transparent, searchable and centrally stored web application, using the MERN technologies of the Fullstack Development – I course. This aim has been achieved. A working MediQueue system was designed, implemented and tested. It lets staff register patients, ranks them by a documented priority score, places them in the most suitable ICU, Emergency or General bed with automatic fallback, releases beds, and keeps an allocation log, through a responsive React interface backed by an Express API and MongoDB.

**Expected results and outcome.** The expected outcome was a system that ranks patients consistently and allocates beds correctly. In testing, the queue was always in descending score order, severe patients received ICU beds first, a severity-5 patient was moved to Emergency when the ICU was full, a patient was left waiting when all 30 beds were taken, invalid input was rejected with clear messages, and released beds became available again.

**Deviations from the expected results and reasons.** (i) Authentication is not enforced: the register and login routes with bcrypt and JWT exist in the backend, but the delivered version has no login screens and its token-verification middleware passes every request. (ii) The scoring weights were chosen by the team for demonstration and have not been validated clinically. (iii) The automated tests used an in-memory store instead of a live MongoDB server and did not drive a browser, so MongoDB-specific behaviour and browser interaction were not tested automatically. (iv) The screens refresh by polling every 5 seconds instead of receiving live updates. (v) Discharged patients stay in the database and there is no archive. (vi) The sample-data and reset routes are not protected.

## 4.2. Future Work

The way ahead consists of: (1) re-enabling login with roles for administrators, doctors and nurses, using the existing bcrypt and JWT module, and protecting every route; (2) live updates with WebSockets or server-sent events; (3) an administration screen to change ward capacities and scoring weights; (4) transfer of patients between wards and an expected discharge time; (5) automated tests kept in the project, with Supertest for the API and a

browser test tool for the interface; (6) deployment to a cloud platform with MongoDB Atlas and HTTPS; (7) Redux Toolkit or the Context API for shared state; (8) audit trails and privacy features for compliance with the Digital Personal Data Protection Act, 2023; and (9) a React Native mobile application that reuses the same API.

# REFERENCES


1. Brown, E. (2019) *Web Development with Node and Express*, 2nd Edn., O'Reilly Media, Sebastopol, CA.

2. Chodorow, K. (2019) *MongoDB: The Definitive Guide*, 3rd Edn., O'Reilly Media, Sebastopol, CA.

3. Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*, PhD dissertation, University of California, Irvine, CA.

4. Fowler, M. (2002) *Patterns of Enterprise Application Architecture*, Addison-Wesley, Boston, MA.

5. Gilboy, N., Tanabe, P., Travers, D. and Rosenau, A.M. (2020) *Emergency Severity Index (ESI): A Triage Tool for Emergency Departments*, Implementation Handbook, 2020 Edn., Emergency Nurses Association, Schaumburg, IL.

6. Government of India (2023) *The Digital Personal Data Protection Act, 2023*, Ministry of Law and Justice, New Delhi.

7. Hahn, E. (2016) *Express in Action*, Manning Publications, Shelter Island, NY.

8. Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force.

9. Lim, G. (2021) *Beginner MERN Stack Developer*, Independently published.

10. Mackway-Jones, K., Marsden, J. and Windle, J. (2014) *Emergency Triage: Manchester Triage Group*, 3rd Edn., Wiley-Blackwell, Chichester.

11. Meta Open Source (2024) *React Documentation*, https://react.dev (accessed October 2026).

12. MongoDB, Inc. (2024) *MongoDB Manual*, https://www.mongodb.com/docs/manual/ (accessed October 2026).

13. Mongoose contributors (2024) *Mongoose Documentation*, https://mongoosejs.com/docs/ (accessed October 2026).

14. National Health Authority (2021) *Ayushman Bharat Digital Mission*, Government of India, https://abdm.gov.in (accessed October 2026).

15. OpenJS Foundation (2024) *Express Documentation*, https://expressjs.com (accessed October 2026).

16. OpenJS Foundation (2024) *Node.js Documentation*, https://nodejs.org/docs/latest/api/ (accessed October 2026).

17. OWASP Foundation (2021) *OWASP Top Ten Web Application Security Risks*, https://owasp.org/www-project-top-ten/ (accessed October 2026).

18. Pressman, R.S. and Maxim, B.R. (2019) *Software Engineering: A Practitioner's Approach*, 9th Edn., McGraw-Hill Education, New York.

19. Provos, N. and Mazières, D. (1999) ‘A Future-Adaptable Password Scheme’, *Proceedings of the USENIX Annual Technical Conference*, Monterey, CA, pp. 81–91.

20. Sommerville, I. (2016) *Software Engineering*, 10th Edn., Pearson Education, Harlow.

21. Subramanian, V. (2019) *Pro MERN Stack*, 2nd Edn., Apress, Berkeley, CA.

# APPENDIX-1: MEDIQUEUE – DATA MODEL AND CORE LOGIC (SOURCE LISTINGS)

This appendix lists the source code of the data model and the core logic of MediQueue, exactly as in the delivered project.

### A1.1 backend/models/Patient.js

```
// Experiment 8: schema with field-level validations and a reference to the Bed model
const mongoose = require('mongoose');


const patientSchema = new mongoose.Schema(
  {
    name: { type: String, required: [true, 'Name is required'], minlength: [3, 'Name must
be at least 3 characters'], trim: true },
    age: { type: Number, required: [true, 'Age is required'], min: [0, 'Age cannot be
negative'], max: [120, 'Age must be 120 or less'] },
    condition: { type: String, required: [true, 'Condition is required'], trim: true },
    severity: { type: Number, required: [true, 'Severity is required'], min: [1,
'Severity must be 1 to 5'], max: [5, 'Severity must be 1 to 5'] },
    distanceKm: { type: Number, default: 0, min: [0, 'Distance cannot be negative'] },
    status: { type: String, enum: { values: ['waiting', 'admitted', 'discharged'],
message: 'Invalid status' }, default: 'waiting' },
    bed: { type: mongoose.Schema.Types.ObjectId, ref: 'Bed', default: null },
    arrivalTime: { type: Date, default: Date.now },
  },
  { timestamps: true }
);


module.exports = mongoose.model('Patient', patientSchema);
```

### A1.2 backend/models/Bed.js

```
const mongoose = require('mongoose');


const bedSchema = new mongoose.Schema({
  bedId: { type: String, required: true, unique: true },
  ward: { type: String, enum: ['ICU', 'Emergency', 'General'], required: true },
  isAvailable: { type: Boolean, default: true },
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'Patient', default: null },
});


module.exports = mongoose.model('Bed', bedSchema);
```

### A1.3 backend/utils/priority.js

```
// Experiment 5: custom Node module (like mathUtils) holding the hospital's priority
rules
const { add, multiply, subtract } = require('./mathUtils');


const SEVERITY_LABELS = { 1: 'Minor', 2: 'Moderate', 3: 'Serious', 4: 'Critical', 5:
'Life-threatening' };


// Higher score = treated sooner
function priorityScore(patient, now = Date.now()) {
  const waitingMinutes = subtract(now, new Date(patient.arrivalTime).getTime()) / 60000;
  const severityScore = multiply(patient.severity, 40);
  const waitingScore = Math.min(waitingMinutes * 0.5, 30);
  const distanceScore = Math.max(0, 20 - (patient.distanceKm || 0) * 0.5);
  const ageScore = patient.age >= 65 || patient.age <= 5 ? 10 : 0;
  return add(add(severityScore, waitingScore), add(distanceScore, ageScore));
}
```

```
// Which wards to try, in order, for a given severity
function wardPreference(severity) {
  if (severity >= 4) return ['ICU', 'Emergency', 'General'];
  if (severity === 3) return ['Emergency', 'ICU', 'General'];
  return ['General', 'Emergency', 'ICU'];
}

const round1 = (n) => Math.round(n * 10) / 10;

module.exports = { SEVERITY_LABELS, priorityScore, wardPreference, round1 };
```

### A1.4 backend/utils/allocate.js

```
// Bed allocation logic: pick the highest-priority waiting patient and give them the best
free bed
const Patient = require('../models/Patient');
const Bed = require('../models/Bed');
const AllocationLog = require('../models/AllocationLog');
const { priorityScore, wardPreference, round1 } = require('./priority');

async function allocateNext() {
  const waiting = await Patient.find({ status: 'waiting' });
  if (waiting.length === 0) return { status: 'empty' };

  const now = Date.now();
  waiting.sort((a, b) => priorityScore(b, now) - priorityScore(a, now));
  const patient = waiting[0];

  for (const ward of wardPreference(patient.severity)) {
    const bed = await Bed.findOne({ ward, isAvailable: true }).sort({ bedId: 1 });
    if (!bed) continue;

    bed.isAvailable = false;
    bed.patient = patient._id;
    await bed.save();

    patient.status = 'admitted';
    patient.bed = bed._id;
    await patient.save();

    const score = round1(priorityScore(patient, now));
    const waitMinutes = round1((now - patient.arrivalTime.getTime()) / 60000);
    await AllocationLog.create({
      patient: patient._id,
      patientName: patient.name,
      severity: patient.severity,
      ward,
      bedId: bed.bedId,
      score,
      waitMinutes,
    });
    return { status: 'allocated', patient: patient.name, ward, bedId: bed.bedId, score };
  }

  return { status: 'no_bed', patient: patient.name };
}

module.exports = { allocateNext };
```

### A1.5 Automated tests

The 44 test cases of Chapter 3 are in the script `run-tests.js` supplied with this report. Placed in `backend/tests/`, it runs with `node tests/run-tests.js` and prints the

result of every case. The remaining source files are included in the project folder submitted with this report.

# USER MANUAL

This manual explains, step by step, how to install, run and use MediQueue on a Windows computer.

## U1. Software required

- Windows 10 or later with an internet connection (needed once to download the npm packages).
- Node.js 18 or later, installed and available on the PATH. Check with `node -v` in a terminal.
- MongoDB Community Server with MongoDB Compass, or a free MongoDB Atlas cluster.
- Visual Studio Code and a web browser such as Chrome, Edge or Firefox.

## U2. Preparing the database

1. Start the MongoDB service (it starts automatically after a normal installation).
2. Open MongoDB Compass and connect to `mongodb://127.0.0.1:27017`. If you use Atlas, copy your connection string instead.
3. Nothing else is needed: the database `mediqueue` and the 30 beds are created automatically the first time the backend starts.

## U3. Configuring the backend

1. Open `backend/.env` in VS Code.
2. Check that `MONGO_URI` points to your MongoDB server (for Atlas, paste the Atlas string).
3. Replace the value of `JWT_SECRET` with any long random text, and save the file.

## U4. Starting the application

1. Open the `mediqueue` folder in VS Code and choose Terminal > New Terminal.
2. In the first terminal type `cd backend`, then `npm install`, then `npm start`. Wait for the messages “MongoDB connected” and “Server running on http://localhost:5000”.

3. Open a second terminal and type `cd frontend`, then `npm install`, then `npm run dev`.
4. Open http://localhost:5173 in the browser. The dashboard appears (Figure 3.2).
5. To stop the application, press Ctrl+C in each terminal.

## U5. Using the application

The menu on the left leads to the Dashboard, Patients, Add patient, Waiting queue and Allocation log pages.

### U5.1. Adding a patient

1. Click **Add patient** in the menu.
2. Enter the patient's name, age and condition, choose the severity from 1 (Minor) to 5 (Life-threatening) and enter the distance from the hospital in kilometres.
3. Click **Add to waiting queue**. If a value is wrong, a message appears above the form (Figure 3.5); correct it and try again.

To load five sample patients quickly, type `node seed.js` in the backend terminal. This clears existing patients and logs and frees all beds before adding the samples.

### U5.2. Viewing the queue and allocating beds

Open **Waiting queue** to see the ranking (Figure 3.3). On the **Dashboard**, click **Allocate next patient** to place the top-ranked patient, or **Allocate all waiting** to place everyone until the beds run out. The message under the buttons shows the patient, bed and score, and the ward bars update (Figure 3.2). The **Allocation log** page keeps the history (Figure 3.6).

### U5.3. Searching, editing, releasing and deleting

On the **Patients** page type part of a name in the search box, or use the severity, status and sort lists (Figure 3.4). Click a patient's name to see the details. Click **Edit**, change the fields in the window and click **Save changes**. Click **Release** on an admitted patient to discharge the patient and free the bed. Click **Delete** and confirm to remove a record. **Reset system** on the dashboard deletes all patients and logs and frees every bed, so use it with care.

## U6. Troubleshooting

If the backend stops with "Could not start server", MongoDB is not running or `MONGO_URI` is wrong; start MongoDB or correct the string. If the pages show "Failed to fetch" or a

network error, make sure the backend is running on port 5000. If a port is busy, close the other program and restart. If `npm install` fails, check the internet connection and the Node.js version.

# ACHIEVEMENTS

- Delivered a complete working hospital bed-management application using React, Express, Node.js, MongoDB and Mongoose.
- Designed an open, documented priority score and a ward-fallback allocation algorithm, and compared two alternative designs with a weighted scoring table.
- Implemented a REST API with 16 endpoints, custom logger, validation and global error-handling middleware, and consistent JSON responses.
- Applied field-level schema validation, references with populate, regular-expression search, sorting and pagination in Mongoose.
- Built a single-page React interface with routing, hooks, controlled forms, automatic refresh and clear empty and error states.
- Wrote and passed 44 automated test cases covering the API, validation rules, priority score, ward fallback and error handling.